

CM2005 Object Oriented Programming Mid Term Project Complete Code

By Ngon Nguyen Below is my complete code for the mid term project of CM2005. This pdf file will include all of the code within my project and their comments. Please also note that inside each source code file and header files, all parts that are personally wrote by me without assistance are preceded by the comment `// ADDITION #num` with `num` being the numbering I assigned to keep track of what I wrote myself, not necessarily their order of creation. The code pasted in here are by alphabetic order of their file name.

Candlestick.cpp:

```
// ADDITION #12 - CANDLESTICK CLASS IMPLEMENTATION FILE
#include "Candlestick.h"

// constructor
Candlestick::Candlestick(std::string timestamp, long double high,
                        long double low, long double open, long double close)
    : timestamp(timestamp), high(high), low(low), open(open), close(close) {}

// getters
std::string Candlestick::getTimestamp() const { return timestamp; }
long double Candlestick::getHigh() const { return high; }
long double Candlestick::getLow() const { return low; }
long double Candlestick::getOpen() const { return open; }
long double Candlestick::getClose() const { return close; }

// setters
void Candlestick::setTimestamp(std::string timestamp) {
    this->timestamp = timestamp;
}
void Candlestick::setHigh(long double high) { this->high = high; }
void Candlestick::setLow(long double low) { this->low = low; }
void Candlestick::setOpen(long double open) { this->open = open; }
void Candlestick::setClose(long double close) { this->close = close; }
void Candlestick::setType(OrderBookType type) { this->type = type; }

// static methods
/** Calculate the ranges for the candlestick (from lowest to highest in the
 * input entries vector) to populate the Y axis of the Candlestick Graph
 * @Params entries: the input candlestick vector, min: lowest price in the input
 * entries, max: the reverse, entry_count: Y axis number of entries
 * @Return a vector of long doubles, each is an entry on the Y axis*/
std::vector<long double>
Candlestick::getIntervalRanges(std::vector<Candlestick> entries,
```

```

        long double min, long double max,
        int entry_count) {

    std::vector<long double> ranges;
    long double range = max - min;
    long double interval = range / entry_count;

    for (int i = entry_count - 1; i >= 0; i--) {
        ranges.push_back(min + (interval * i));
    }

    return ranges;
}

/** Get the lowest price in the input entries vector
 * @Params entries: the input candlestick vector
 * @Return the lowest price in the input entries vector*/
long double Candlestick::getMin(std::vector<Candlestick> entries) {
    long double min = entries[0].low;
    for (auto const &entry : entries) {
        if (entry.low < min) {
            min = entry.low;
        }
    }
    return min;
}

/** Get the highest price in the input entries vector
 * @Params entries: the input candlestick vector
 * @Return the highest price in the input entries vector*/
long double Candlestick::getMax(std::vector<Candlestick> entries) {
    long double max = entries[0].high;
    for (auto const &entry : entries) {
        if (entry.high > max) {
            max = entry.high;
        }
    }
    return max;
}
// END OF STUDENT CODE

Candlestick.h:

// ADDITION #10 - CANDLESTICK CLASS HEADER FILE
// This file contains the header file for the Candlestick class, which is used
// to represent a candlestick in the candlestick chart. A candlestick object
// includes its timestamp, open, high, low, close, and type (bid/ask).

```

```

#pragma once

#include "OrderBookEntry.h"
#include <vector>

class Candlestick {
private:
    std::string timestamp;
    long double open;
    long double high;
    long double low;
    long double close;
    OrderBookType type;

public:
    // constructors
    Candlestick(std::string timestamp, long double open, long double high,
                long double low, long double close);

    // getters
    std::string getTimestamp() const;
    long double getOpen() const;
    long double getHigh() const;
    long double getLow() const;
    long double getClose() const;
    OrderBookType getType() const;

    // setters
    void setTimestamp(std::string timestamp);
    void setOpen(long double open);
    void setHigh(long double high);
    void setLow(long double low);
    void setClose(long double close);
    void setType(OrderBookType type);

    // static methods
    static std::vector<long double>
    getIntervalRanges(std::vector<Candlestick> entries, long double min,
                      long double max, int entry_count);
    static long double getMin(std::vector<Candlestick> entries);
    static long double getMax(std::vector<Candlestick> entries);
};
// END OF STUDENT CODE

Coord.h:

// ADDITION #14 - COORD CLASS IMPLEMENTATION FILE

```

```

#include "Coord.h"

// constructor
Coord::Coord(int x, int y) : x(x), y(y) {}

// getters
int Coord::getX() const { return x; }
int Coord::getY() const { return y; }

// setters
void Coord::setX(int x) { this->x = x; }
void Coord::setY(int y) { this->y = y; }

// static methods
/** Function to convert a vector of candlesticks to a vector of coordinates
 * ready to be drawn on the CandlestickGraph
 * @Params entries: the input candlestick vector to be converted,
 * minPointY: the Coord(ination) object of the lowest entry on Y axis
 * maxPointY: self-explanatory, reverse of minPointY
 * maxEntries: the maximum amount of entries possible that can be drawn
 * @Returns a vector of coordinations ready to be drawn */
std::vector<std::vector<long double>> Coord::convertCandlestickToCoord(
    std::vector<Candlestick> entries, Coord minPointY, Coord maxPointY,
    long double minEntries, long double maxEntries) {
    std::vector<std::vector<long double>> ycoords;
    for (auto const &entry : entries) {
        std::vector<long double> points;
        long double yHigh = mapping(entry.getHigh(), minEntries, maxEntries,
                                     maxPointY.getY(), minPointY.getY());
        long double yLow = mapping(entry.getLow(), minEntries, maxEntries,
                                    maxPointY.getY(), minPointY.getY());
        long double yOpen = mapping(entry.getOpen(), minEntries, maxEntries,
                                    maxPointY.getY(), minPointY.getY());
        long double yClose = mapping(entry.getClose(), minEntries, maxEntries,
                                      maxPointY.getY(), minPointY.getY());

        points.push_back(yHigh);
        points.push_back(yLow);
        points.push_back(yOpen);
        points.push_back(yClose);
        ycoords.push_back(points);
    }
    return ycoords;
}
// END OF STUDENT CODE

```

CSVReader.cpp:

```

#include "CSVReader.h"
#include <fstream>
#include <iostream>

CSVReader::CSVReader() {}

std::vector<OrderBookEntry> CSVReader::readCSV(std::string csvFilename) {
    std::vector<OrderBookEntry> entries;

    std::ifstream csvFile{csvFilename};
    std::string line;
    if (csvFile.is_open()) {
        while (std::getline(csvFile, line)) {
            try {
                OrderBookEntry obe = stringsToOBE(tokenise(line, ','));
                entries.push_back(obe);
            } catch (const std::exception &e) {
                std::cout << "CSVReader::readCSV bad data" << std::endl;
            }
        } // end of while
    }

    std::cout << "CSVReader::readCSV read " << entries.size() << " entries"
                << std::endl;
    return entries;
}

std::vector<std::string> CSVReader::tokenise(std::string csvLine,
                                             char separator) {

    std::vector<std::string> tokens;
    signed int start, end;
    std::string token;
    start = csvLine.find_first_not_of(separator, 0);
    do {
        end = csvLine.find_first_of(separator, start);
        if (start == csvLine.length() || start == end)
            break;
        if (end >= 0)
            token = csvLine.substr(start, end - start);
        else
            token = csvLine.substr(start, csvLine.length() - start);
        tokens.push_back(token);
        start = end + 1;
    } while (end > 0);

    return tokens;
}

```

```

}

OrderBookEntry CSVReader::stringsToOBE(std::vector<std::string> tokens) {
    double price, amount;

    if (tokens.size() != 5) // bad
    {
        std::cout << "Bad line " << std::endl;
        throw std::exception{};
    }
    // we have 5 tokens
    try {
        price = std::stod(tokens[3]);
        amount = std::stod(tokens[4]);
    } catch (const std::exception &e) {
        std::cout << "CSVReader::stringsToOBE Bad float! " << tokens[3]
                  << std::endl;
        std::cout << "CSVReader::stringsToOBE Bad float! " << tokens[4]
                  << std::endl;
        throw;
    }

    OrderBookEntry obe{price, amount, tokens[0], tokens[1],
                       OrderBookEntry::stringToOrderBookType(tokens[2])};

    return obe;
}

OrderBookEntry CSVReader::stringsToOBE(std::string priceString,
                                         std::string amountString,
                                         std::string timestamp,
                                         std::string product,
                                         OrderBookType orderType) {

    double price, amount;
    try {
        price = std::stod(priceString);
        amount = std::stod(amountString);
    } catch (const std::exception &e) {
        std::cout << "CSVReader::stringsToOBE Bad float! " << priceString
                  << std::endl;
        std::cout << "CSVReader::stringsToOBE Bad float! " << amountString
                  << std::endl;
        throw;
    }
    OrderBookEntry obe{price, amount, timestamp, product, orderType};
}

```

```

    return obe;
}'''

```

CSVReader.h:

```

#pragma once

#include "OrderBookEntry.h" #include #include

class CSVReader { public: CSVReader();

    static std::vector<OrderBookEntry> readCSV(std::string csvFile);
    static std::vector<std::string> tokenise(std::string csvLine, char separator);

    static OrderBookEntry stringsToOBE(std::string price,
                                        std::string amount,
                                        std::string timestamp,
                                        std::string product,
                                        OrderBookType OrderBookType);

private:
    static OrderBookEntry stringsToOBE(std::vector<std::string> strings);

};

```

Helpers.cpp:

```

// ADDITION #16 - HELPER FUNCTIONS IMPLEMENTATION FILE //
This is a file that contains the implementation of assorted helper functions //
that doesn't fit in any classes of the project. My first instinct was to make //
this a class as well. However, I decided against it as those helper functions //
are used quite widely throughout the project and creating a Helper object //
everytime I want something quick and simple isn't an efficient use of memory. //
I also don't want to create a class full of static objects only, it just // feels wrong
to create an empty class with only static objects, and seem // overly complicated
while I can achieve the same effect with just function // prototypes. Hence, this
file. #include "Helpers.h" #include

/** Function to tokenise timestamp by the space between the date and time,
with * an added check for empty input, @Params a string of raw timestamp,
@Returns a * 2-element vector, with date and time, ready to be written onto
the X axis*/ std::vector processTimestamp(std::string raw_timestamp) {

std::vector processed_timestamps = {"1970/01/01", "00:00:00.00"}; // tokenise
if not raw_timestamp not null, else return default if (raw_timestamp != "")
{ processed_timestamps = CSVReader::tokenise(raw_timestamp, ' '); } return
processed_timestamps; }

/** Function to return a string of a single character at a given index, @Params
* an index and a string, @Returns a string of a single character at the given

```

```

* index*/ std::string typewriter(int index, std::string text) { std::string str(1,
text[index]); return str; }

/** Function to map a value from one range to another, @Params a input value,
* the input range, and the output range, @Returns a value mapped to the output
* range/ long double mapping(long double val, long double i_min, long double
i_max, long double o_min, long double o_max) { // This algorithm took me a
while to figure out, mostly due to the offset of // -i_min and +o_min return
(val - i_min) (o_max - o_min) / (i_max - i_min) + o_min; }

/** Function to round a long double to the nearest floor long double, @Params
a * long double, * @Returns a vector of long doubles: the floored long double
and the * remainder*/ std::vector processMappingUpper(long double value) {
long double rounded = floor(value); long double difference = value - rounded;
return {rounded, difference}; }

/** Function to round a long double to the nearest ceiling long double, @Params
* a long double, * @Returns a vector of long doubles: the ceiled long double and
the remainder*/ std::vector processMappingLower(long double value) { long
double rounded = ceil(value); long double difference = rounded - value; return
{rounded, difference}; }

/** Function to process the remainder of a value into appropriate candlestick *
shapes on the top and bottom of the candlestick column, @Params a long double
* and a string of the position, @Returns a string of the appropriate unicode
* character*/ std::string processRemainder(long double remainder, std::string
position) { // The partials vector contains the partial values of the candlestick
shapes std::vector partials{1.0 / 8, 1.0 / 4, 3.0 / 8, 1.0 / 2, 5.0 / 8, 3.0 / 4, 7.0
/ 8};

// The column_upper and column_lower vectors contain the unicode charac-
ters // for the candlestick shapes according to partials' index std::vector col-
umn_upper{” // truncated Unicode glyphs because the pdf engine couldn't
handle it

std::vector column_lower{” // truncated Unicode glyphs because the pdf engine
couldn't handle it

// Iterate through the partials vector, and return the appropriate unicode //
character if the remainder is smaller than the partial value if (position ==
“lower”) { for (int i = 0; i < partials.size(); i++) { if (remainder <= partials[i])
{ return column_lower[i]; } } } else { for (int i = 0; i < partials.size(); i++) { if
(remainder <= partials[i]) { return column_upper[i]; } } } // If the remainder is
larger than 7/8, return the full block unicode // character return ” // truncated
Unicode glyphs because the pdf engine couldn't handle it }

/** Function to shorten a number to a more readable format, @Params a num-
ber, * @Returns a string of the shortened number*/ std::string shortenNum-
ber(long long number) { // This algorithm is based on the metric unit prefixes,
with the addition of // decimal places for numbers that are not multiples of

```

```
1000 const std::vector suffixes{"", "K", "M", "B"}; const std::vector values{1LL,
1000LL, 1000000LL, 1000000000LL};
```

```
// Iterate through the values vector, starting from the largest value for (int i
= 3; i >= 0; i--) { // If the number is larger than the current value, divide
the number by the // value and return the quotient with the appropriate suf-
fix // If the number is not a multiple of the value, add a decimal place and
// append the first 3 digits of the remainder if (number >= values[i]) { long
long quotient = number / values[i]; long long remainder = number % values[i];
std::string quotient_str = std::to_string(quotient); std::string remainder_str =
std::to_string(remainder); if (remainder != 0) { quotient_str += "." + remain-
der_str.substr(0, 3); } // Append the suffix and return the string quotient_str
+= suffixes[i]; return quotient_str; } } // If the number is smaller than 1000,
return the number as a string return std::to_string(number); }
```

```
/** Function to return a colored string, @Params text: a string to be colored,
* color: a string of the color to be used, * @Returns a string with the color
code appened to the front and the reset code * to the back*/ std::string col-
orize(std::string text, std::string color) { std::string color_code = ""; if (color
=="red") { color_code = "\033[31m"; } else if (color == "green") { color_code
="\033[32m"; } else if (color == "yellow") { color_code = "\033[33m"; }
else if (color == "blue") { color_code = "\033[34m"; } else if (color == "ma-
genta") { color_code = "\033[35m"; } else if (color == "cyan") { color_code
="\033[36m"; } else if (color == "white") { color_code = "\033[37m"; } return
color_code + text + "\033[0m"; // END OF STUDENT CODE }“
```

Helpers.h:

```
// ADDITION #15 - HELPER FUNCTIONS HEADER FILE
```

```
#pragma once
```

```
#include "CSVReader.h"
```

```
#include <iomanip>
```

```
#include <string>
```

```
#include <vector>
```

```
// Function prototypes
```

```
std::vector<std::string> processTimestamp(std::string raw_timestamp);
```

```
std::string typewriter(int index, std::string text);
```

```
long double mapping(long double val, long double i_min, long double i_max,
                    long double o_min, long double o_max);
```

```
std::vector<long double> processMappingUpper(long double value);
```

```
std::vector<long double> processMappingLower(long double value);
```

```
std::string processRemainder(long double remainder, std::string position);
```

```
std::string shortenNumber(long long number);
```

```
std::string colorize(std::string text, std::string color);
```

```
// END OF STUDENT CODE
```

main.cpp:

```
#include "Wallet.h"
#include <iostream>
#include "MerkelMain.h"
```

```
int main()
{
    MerkelMain app{};
    app.init();
}
```

MerkelMain.cpp:

```
#include "MerkelMain.h"
#include "CSVReader.h"
#include "Candlestick.h"
#include "CandlestickGraph.h"
#include "OrderBookEntry.h"
#include "VolumeGraph.h"
#include <iostream>
#include <vector>
```

```
MerkelMain::MerkelMain() {}
```

```
void MerkelMain::init() {
    int input;
    currentTime = orderBook.getEarliestTime();

    wallet.insertCurrency("BTC", 10);

    while (true) {
        printMenu();
        input = getUserOption();
        processUserOption(input);
        lastTime = orderBook.getLastTime(currentTime);
    }
}
```

```
// ASCII art made using
```

```
// https://patorjk.com/software/taag/#p=display&f=Big%20Money-nw&t=Merkelrex
```

```
void MerkelMain::printMenu() {
```

```
    std::cout << "=====
    "=====
    << std::endl;
```



```

        << std::endl;
    }

void MerkelMain::printMarketStats() {

    for (std::string const &p : orderBook.getKnownProducts()) {
        std::cout << "Product: " << p << std::endl;
        std::vector<OrderBookEntry> entries_ask =
            orderBook.getOrders(OrderBookType::ask, p, currentTime);
        std::vector<OrderBookEntry> entries_ask_last =
            orderBook.getOrders(OrderBookType::ask, p, lastTime);
        std::vector<OrderBookEntry> entries_bid =
            orderBook.getOrders(OrderBookType::bid, p, currentTime);
        std::vector<OrderBookEntry> entries_bid_last =
            orderBook.getOrders(OrderBookType::bid, p, lastTime);

        std::cout << "Asks seen: " << entries_ask.size() << std::endl;
        std::cout << "Max ask: " << OrderBook::getHighPrice(entries_ask)
            << std::endl;
        std::cout << "Min ask: " << OrderBook::getLowPrice(entries_ask)
            << std::endl;
        std::cout << "Opening ask: "
            << OrderBook::getOpeningAndClosingPrice(entries_ask_last)
            << std::endl;
        std::cout << "Closing ask: "
            << OrderBook::getOpeningAndClosingPrice(entries_ask) << std::endl;

        std::cout << "Bids seen: " << entries_bid.size() << std::endl;
        std::cout << "Max bid: " << OrderBook::getHighPrice(entries_bid)
            << std::endl;
        std::cout << "Min bid: " << OrderBook::getLowPrice(entries_bid)
            << std::endl;
        std::cout << "Opening bid: "
            << OrderBook::getOpeningAndClosingPrice(entries_bid_last)
            << std::endl;
        std::cout << "Closing bid: "
            << OrderBook::getOpeningAndClosingPrice(entries_bid) << std::endl;
    }
}

void MerkelMain::enterAsk() {
    std::cout << "Make an ask - enter the amount: product,price, amount, eg "
        "ETH/BTC,200,0.5"
        << std::endl;
    std::string input;
    std::getline(std::cin, input);
}

```

```

std::vector<std::string> tokens = CSVReader::tokenise(input, ',');
if (tokens.size() != 3) {
    std::cout << "MerkelMain::enterAsk Bad input! " << input << std::endl;
} else {
    try {
        OrderBookEntry obe = CSVReader::stringsToOBE(
            tokens[1], tokens[2], currentTime, tokens[0], OrderBookType::ask);
        obe.username = "simuser";
        if (wallet.canFulfillOrder(obe)) {
            std::cout << "Wallet looks good. " << std::endl;
            orderBook.insertOrder(obe);
        } else {
            std::cout << "Wallet has insufficient funds . " << std::endl;
        }
    } catch (const std::exception &e) {
        std::cout << "MerkelMain::enterAsk Bad input " << std::endl;
    }
}
}

void MerkelMain::enterBid() {
    std::cout << "Make an bid - enter the amount: product,price, amount, eg "
        << "ETH/BTC,200,0.5"
        << std::endl;
    std::string input;
    std::getline(std::cin, input);

    std::vector<std::string> tokens = CSVReader::tokenise(input, ',');
    if (tokens.size() != 3) {
        std::cout << "MerkelMain::enterBid Bad input! " << input << std::endl;
    } else {
        try {
            OrderBookEntry obe = CSVReader::stringsToOBE(
                tokens[1], tokens[2], currentTime, tokens[0], OrderBookType::bid);
            obe.username = "simuser";

            if (wallet.canFulfillOrder(obe)) {
                std::cout << "Wallet looks good. " << std::endl;
                orderBook.insertOrder(obe);
            } else {
                std::cout << "Wallet has insufficient funds . " << std::endl;
            }
        } catch (const std::exception &e) {
            std::cout << "MerkelMain::enterBid Bad input " << std::endl;
        }
    }
}

```

```

    }
}

void MerkelMain::printWallet() { std::cout << wallet.toString() << std::endl; }

// #ADDITION #7
void MerkelMain::drawCandlesticks() {
    // get product and type input
    std::cout << "Enter the product / type pair"
                << "draw - eg: ETH/BTC,ask"
                << std::endl;
    std::string input;
    std::getline(std::cin, input);

    // tokenise input by the comma, and check if the size is 2, if not throw error
    std::vector<std::string> tokens = CSVReader::tokenise(input, ',');
    if (tokens.size() != 2) {
        std::cout << "MerkelMain::drawCandlesticks Bad input! " << input
                  << std::endl;
    } else {
        // try to calculate and draw candlestick
        try {
            // calculate candlestick from the current timestamp
            std::vector<Candlestick> candlesticks =
                orderBook.computeCandlesticks(tokens[0], tokens[1], currentTime);
            // prevent segfault if there is no data before the time or bad input
            if (candlesticks.empty()) {
                std::cout
                    << "WARNING!!! No candlesticks to draw. Either there is no data \n"
                    << "before the current time or you misspelled the product!!!"
                    << std::endl;
                return;
            } else {
                // check if the size of candlesticks is more than 8 (max amount of
                // candlesticks on the graph) i.e. when advance time
                if (candlesticks.size() > 8) {
                    // delete the first entry of candlesticks until the size is 8
                    while (candlesticks.size() > 8) {
                        candlesticks.erase(candlesticks.begin());
                    }
                }
                // draw candlestick using the CandlestickGraph class
                CandlestickGraph graph(candlesticks);
                graph.prep();
                graph.print();
            }
        }
    }
}

```

```

    } catch (const std::exception &e) {
        std::cout << " MerkelMain::drawCandlesticks Bad input " << std::endl;
    }
}
// END OF STUDENT CODE

// #ADDITION #8
// compute and then draw the volume graph, based on the current timestamp, using
// the VolumeGraph class
void MerkelMain::drawVolumeGraph() {
    std::vector<Volume> volumes = orderBook.computeVolumes(currentTime);
    VolumeGraph graph(volumes);
    graph.prep();
    graph.print();
}
// END OF STUDENT CODE

void MerkelMain::gotoNextTimeframe() {
    std::cout << "Going to next time frame. " << std::endl;
    for (std::string p : orderBook.getKnownProducts()) {
        std::cout << "matching " << p << std::endl;
        std::vector<OrderBookEntry> sales =
            orderBook.matchAsksToBids(p, currentTime);
        std::cout << "Sales: " << sales.size() << std::endl;
        for (OrderBookEntry &sale : sales) {
            std::cout << "Sale price: " << sale.price << " amount " << sale.amount
                << std::endl;
            if (sale.username == "simuser") {
                // update the wallet
                wallet.processSale(sale);
            }
        }
    }
    currentTime = orderBook.getNextTime(currentTime);
}

int MerkelMain::getUserOption() {
    int userOption = 0;
    std::string line;
    std::cout << "Type in 1-8" << std::endl;
    std::getline(std::cin, line);
    try {
        userOption = std::stoi(line);
    } catch (const std::exception &e) {
        //
    }
}

```

```

    }
    std::cout << "You chose: " << userOption << std::endl;
    std::cout << "=====
    "
    << std::endl;
    return userOption;
}

void MerkelMain::processUserOption(int userOption) {
    if (userOption == 0) // bad input
    {
        std::cout << "Invalid choice. Choose 1-8" << std::endl;
    }
    if (userOption == 1) {
        printHelp();
    }
    if (userOption == 2) {
        printMarketStats();
    }
    if (userOption == 3) {
        enterAsk();
    }
    if (userOption == 4) {
        enterBid();
    }
    if (userOption == 5) {
        printWallet();
    }
    if (userOption == 6) {
        drawCandlesticks();
    }
    if (userOption == 7) {
        drawVolumeGraph();
    }
    if (userOption == 8) {
        gotoNextTimeframe();
    }
}

MerkelMain.h:

#pragma once

#include "OrderBook.h"
#include "OrderBookEntry.h"
#include "Wallet.h"
#include <vector>

```

```

class MerkelMain {
public:
    MerkelMain();
    /** Call this to start the sim */
    void init();

private:
    void printMenu();
    void printHelp();
    void printMarketStats();
    void enterAsk();
    void enterBid();
    void printWallet();
    void drawCandlesticks();
    void drawVolumeGraph();
    void gotoNextTimeframe();
    int getUserOption();
    void processUserOption(int userOption);

    std::string currentTime;
    std::string lastTime;

    // OrderBook orderBook{"20200317.csv"};
    OrderBook orderBook{"20200601.csv"};
    Wallet wallet;
};

OrderBook.cpp:

#include "OrderBook.h"
#include "CSVReader.h"
#include "Candlestick.h"
#include <algorithm>
#include <chrono>
#include <iostream>
#include <map>
#include <thread>

/** construct, reading a csv data file */
OrderBook::OrderBook(std::string filename) {
    orders = CSVReader::readCSV(filename);
}

/** return vector of all know products in the dataset*/
std::vector<std::string> OrderBook::getKnownProducts() {
    std::vector<std::string> products;

```

```

std::map<std::string, bool> prodMap;

for (OrderBookEntry &e : orders) {
    prodMap[e.product] = true;
}

// now flatten the map to a vector of strings
for (auto const &e : prodMap) {
    products.push_back(e.first);
}

return products;
}

// ADDITION #4
/** return vector of all individual products in the dataset*/
std::vector<std::string> OrderBook::getIndividualProducts() {
    // use the built-in getKnownProducts() function to get all known products
    std::vector<std::string> possibleProducts = OrderBook::getKnownProducts();
    std::vector<std::string> individualProducts;
    for (std::string &p : possibleProducts) {
        std::string product = CSVReader::tokenise(p, '/') [0];
        // check if the product is already in the vector, if not, add it
        if (std::find(individualProducts.begin(), individualProducts.end(),
            product) == individualProducts.end()) {
            individualProducts.push_back(product);
        }
    }
    return individualProducts;
}

// END OF STUDENT CODE

// ADDITION #9
/** return vector of all secondary products (the second product in the product
 * pair) in the dataset*/
std::vector<std::string> OrderBook::getSecondaryProducts() {
    // use the built-in getKnownProducts() function to get all known products
    std::vector<std::string> possibleProducts = OrderBook::getKnownProducts();
    std::vector<std::string> secondaryProducts;
    for (std::string &p : possibleProducts) {
        std::string product = CSVReader::tokenise(p, '/') [1];
        // check if the product is already in the vector, if not, add it
        if (std::find(secondaryProducts.begin(), secondaryProducts.end(),
            product) == secondaryProducts.end() &&
            product != "USDT") {

```

```

        secondaryProducts.push_back(product);
    }
}
return secondaryProducts;
}
// END OF STUDENT CODE

/** return vector of Orders according to the sent filters*/
std::vector<OrderBookEntry> OrderBook::getOrders(OrderBookType type,
                                                std::string product,
                                                std::string timestamp) {

    std::vector<OrderBookEntry> orders_sub;
    for (OrderBookEntry &e : orders) {
        if (e.orderType == type && e.product == product &&
            e.timestamp == timestamp) {
            orders_sub.push_back(e);
        }
    }
    return orders_sub;
}

/** returns the highest price in a passed in vector of orders*/
long double OrderBook::getHighPrice(std::vector<OrderBookEntry> &orders) {
    if (orders.empty()) {
        return 0; // Return 0 if the input vector is empty
    }
    long double max = orders[0].price;
    for (OrderBookEntry &e : orders) {
        if (e.price > max)
            max = e.price;
    }
    return max;
}

/** returns the lowest price in a passed in vector of orders*/
long double OrderBook::getLowPrice(std::vector<OrderBookEntry> &orders) {
    if (orders.empty()) {
        return 0; // Return 0 if the input vector is empty
    }
    long double min = orders[0].price;
    for (OrderBookEntry &e : orders) {
        if (e.price < min)
            min = e.price;
    }
    return min;
}

```

```

// ADDITION #2
/** returns the opening OR closed price in a passed in vector of orders
 * If there is nothing in the vector of orders, return empty string */
long double
OrderBook::getOpeningAndClosingPrice(std::vector<OrderBookEntry> &orders) {
    if (orders.empty()) {
        return 0; // Return 0 if the input vector is empty
    }
    long double openingPrice = 0;
    long double weightedTotal = 0;
    long double totalAmount = 0;
    // Loop through the vector of orders and calculate the weighted total
    // (amount * price) and total amount
    for (OrderBookEntry &e : orders) {
        totalAmount += e.amount;
        weightedTotal += e.amount * e.price;
    }
    // Return the weighted total divided by the total amount, avoid division by 0
    // by using a ternary operator
    return totalAmount != 0 ? weightedTotal / totalAmount : 0;
}

std::string OrderBook::getEarliestTime() { return orders[0].timestamp; }

std::string OrderBook::getNextTime(std::string timestamp) {
    std::string next_timestamp = "";
    for (OrderBookEntry &e : orders) {
        if (e.timestamp > timestamp) {
            next_timestamp = e.timestamp;
            break;
        }
    }
    if (next_timestamp == "") {
        next_timestamp = orders[0].timestamp;
    }
    return next_timestamp;
}

// END OF STUDENT CODE

// ADDITION #1
/** returns the previous timestamp before the input timestamp*/
std::string OrderBook::getLastTime(std::string timestamp) {
    std::string last_timestamp = "";
    // This reverse loop was a pain to figure out
    for (auto e = orders.size() - 1; e > 0; e--) {

```

```

        // If the timestamp is immediately less than the input timestamp, set the
        // last timestamp, break out of the loop and return it
        if (orders[e].timestamp < timestamp) {
            last_timestamp = orders[e].timestamp;
            break;
        }
    }
    return last_timestamp;
}
// END OF STUDENT CODE

// ADDITION #3 --- SATISFYING TASK 1 REQUIREMENTS ---
/** returns a vector of Candlesticks data to be drawn as a graph
 * requires @params: a pair of product names, a type (bid/ask) and the current
 * timestamp*/
std::vector<Candlestick>
OrderBook::computeCandlesticks(std::string product, std::string type,
                                std::string currentTime) {

    std::vector<Candlestick> candlesticks_sub;
    std::vector<OrderBookEntry> orders_sub;
    std::string lastTime = OrderBook::getEarliestTime();
    std::string endTime = OrderBook::getNextTime(currentTime);

    // Loop through the orders vector and
    // find the orders that match the timestamp
    for (auto i = 0; i < orders.size(); i++) {
        if (orders[i].timestamp == currentTime) {
            // Then once found, create a sub vector of orders from the beginning of
            // time to the current time, we will use this vector from now on to save
            // computational resources (why calculating the whole order vector if we
            // just need data from the past to a certain point? The software isn't
            // suppose to "see" into the future, right?)
            orders_sub.assign(orders.begin(), orders.begin() + i);
        }
    }

    // Traverse the vector we got above, if the timestamp is different from the
    // last timestamp, get all the orders (ask or bid depended on the input type)
    // for that timestamp and the last timestamp
    for (OrderBookEntry &o : orders_sub) {
        if (o.timestamp != lastTime) {
            std::vector<OrderBookEntry> orders = OrderBook::getOrders(
                type == "ask" ? OrderBookType::ask : OrderBookType::bid, product,
                o.timestamp);
            std::vector<OrderBookEntry> orders_last = OrderBook::getOrders(

```

```

        type == "ask" ? OrderBookType::ask : OrderBookType::bid, product,
        lastTime);
    // calculate the open, high, low and close prices for the candlestick and
    // use them to create a candlestick object
    long double open = OrderBook::getOpeningAndClosingPrice(orders_last);
    long double high = OrderBook::getHighPrice(orders);
    long double low = OrderBook::getLowPrice(orders);
    long double close = OrderBook::getOpeningAndClosingPrice(orders);
    Candlestick candlestick(o.timestamp, high, low, open, close);
    // only add candlestick if it is valid (no 0 values, as a crypto can't be
    // 0 because at that point it will be delisted and cease to exist)
    if (candlestick.getOpen() != 0 && candlestick.getHigh() != 0 &&
        candlestick.getLow() != 0 && candlestick.getClose() != 0) {
        candlesticks_sub.push_back(candlestick);
    }
    // set the last timestamp to the current timestamp
    lastTime = o.timestamp;
}
}
// return the vector of candlesticks
return candlesticks_sub;
}
// END OF STUDENT CODE

// ADDITION #5
/** returns a vector of Volume data to be drawn as a graph
 * requires @params: the current timestamp*/
std::vector<Volume> OrderBook::computeVolumes(std::string currentTime) {

    std::vector<Volume> volumes_sub;
    std::vector<Volume> secondaryVolumes_sub;

    std::vector<std::string> individualProducts =
        OrderBook::getIndividualProducts();
    std::vector<std::string> secondaryProducts =
        OrderBook::getSecondaryProducts();

    // For each individual products (ETH, BTC, etc.)
    for (auto &p : individualProducts) {

        std::vector<OrderBookEntry> orders_sub;
        double volumeUSDT = 0;
        double secondaryVolumes = 0;

        // Loop through the orders vector and find the orders that match the product
        for (OrderBookEntry &order : orders) {

```

```

        std::string firstProduct = CSVReader::tokenise(order.product, '/') [0];
        if (firstProduct == p && order.timestamp == currentTime) {
            orders_sub.push_back(order);
        }
    }

    // Traverse the vector we got above and calculate the volume in USDT if the
    // second part of the pair is USDT
    for (OrderBookEntry &order : orders_sub) {
        std::string secondProduct = CSVReader::tokenise(order.product, '/') [1];
        if (secondProduct == "USDT") {
            volumeUSDT += order.amount * order.price;
        } else {
            // if the second part of the pair is not USDT (mainly BTC in the
            // provided dataset), we need to find the average price of the secondary
            // product, and multiply it by the amount and price of the order,
            // basically converting it to USDT and then add it to the volume
            double priceInUSDT =
                OrderBook::averagePrice(secondProduct + "/USDT", currentTime);
            double volumeInUSDT = order.amount * order.price * priceInUSDT;
            volumeUSDT += volumeInUSDT;
            // ALSO, we store this secondary volume, as I count the secondary volume
            // as a volume of the secondary product (BTC in this case)
            secondaryVolumes_sub.push_back(Volume(secondProduct, volumeInUSDT));
        }
    }
    volumes_sub.push_back(Volume(p, volumeUSDT));
}

// traverse through secondaryVolumes_sub , if the product is found in the
// volumes_sub vector, add the volume to the volume of the product
for (auto &v : secondaryVolumes_sub) {
    for (auto &v2 : volumes_sub) {
        if (v.getProduct() == v2.getProduct()) {
            v2.setVolume(v2.getVolume() + v.getVolume());
        }
    }
}

return volumes_sub;
}
// END OF STUDENT CODE

// ADDITION #6
/** returns the average price of a product at a given timestamp
 * requires @params: the product and the timestamp*/

```

```

double OrderBook::averagePrice(std::string product, std::string timestamp) {
    double totalPrice = 0;
    double totalAmount = 0;
    for (OrderBookEntry &order : orders) {
        if (order.product == product && order.timestamp == timestamp) {
            // calculate the total price by adding the individual order
            // (amount * price) divide by total amount
            totalPrice += (order.amount * order.price);
            totalAmount += order.amount;
        }
    }
    // return 0 if totalAmount is 0 to avoid division by 0, else return the
    // average price
    return totalAmount == 0 ? 0 : totalPrice / totalAmount;
}
// END OF STUDENT CODE

void OrderBook::insertOrder(OrderBookEntry &order) {
    orders.push_back(order);
    std::sort(orders.begin(), orders.end(), OrderBookEntry::compareByTimestamp);
}

std::vector<OrderBookEntry> OrderBook::matchAsksToBids(std::string product,
                                                        std::string timestamp) {

    // asks = orderbook.asks
    std::vector<OrderBookEntry> asks =
        getOrders(OrderBookType::ask, product, timestamp);
    // bids = orderbook.bids
    std::vector<OrderBookEntry> bids =
        getOrders(OrderBookType::bid, product, timestamp);

    // sales = []
    std::vector<OrderBookEntry> sales;

    // I put in a little check to ensure we have bids and asks
    // to process.
    if (asks.size() == 0 || bids.size() == 0) {
        std::cout << " OrderBook::matchAsksToBids no bids or asks" << std::endl;
        return sales;
    }

    // sort asks lowest first
    std::sort(asks.begin(), asks.end(), OrderBookEntry::compareByPriceAsc);
    // sort bids highest first
    std::sort(bids.begin(), bids.end(), OrderBookEntry::compareByPriceDesc);
    // for ask in asks:

```

```

std::cout << "max ask " << asks[asks.size() - 1].price << std::endl;
std::cout << "min ask " << asks[0].price << std::endl;
std::cout << "max bid " << bids[0].price << std::endl;
std::cout << "min bid " << bids[bids.size() - 1].price << std::endl;

for (OrderBookEntry &ask : asks) {
    //     for bid in bids:
    for (OrderBookEntry &bid : bids) {
        //         if bid.price >= ask.price # we have a match
        if (bid.price >= ask.price) {
            //             sale = new order()
            //             sale.price = ask.price
            OrderBookEntry sale{ask.price, 0, timestamp, product,
                                OrderBookType::asksale};

            if (bid.username == "simuser") {
                sale.username = "simuser";
                sale.orderType = OrderBookType::bidsale;
            }
            if (ask.username == "simuser") {
                sale.username = "simuser";
                sale.orderType = OrderBookType::asksale;
            }
        }

        //             # now work out how much was sold and
        //             # create new bids and asks covering
        //             # anything that was not sold
        //             if bid.amount == ask.amount: # bid completely clears
        //             ask
        if (bid.amount == ask.amount) {
            //                 sale.amount = ask.amount
            sale.amount = ask.amount;
            //                 sales.append(sale)
            sales.push_back(sale);
            //                 bid.amount = 0 # make sure the bid is not
            //                 processed again
            bid.amount = 0;
            //                 # can do no more with this ask
            //                 # go onto the next ask
            //                 break
            break;
        }
        //             if bid.amount > ask.amount: # ask is completely gone
        //             slice the bid
        if (bid.amount > ask.amount) {
            //                 sale.amount = ask.amount

```

```

        sale.amount = ask.amount;
        //          sales.append(sale)
        sales.push_back(sale);
        //          # we adjust the bid in place
        //          # so it can be used to process the next ask
        //          bid.amount = bid.amount - ask.amount
        bid.amount = bid.amount - ask.amount;
        //          # ask is completely gone, so go to next ask
        //          break
        break;
    }

    //          if bid.amount < ask.amount # bid is completely gone,
    //          slice the ask
    if (bid.amount < ask.amount && bid.amount > 0) {
        //          sale.amount = bid.amount
        sale.amount = bid.amount;
        //          sales.append(sale)
        sales.push_back(sale);
        //          # update the ask
        //          # and allow further bids to process the remaining
        //          amount ask.amount = ask.amount - bid.amount
        ask.amount = ask.amount - bid.amount;
        //          bid.amount = 0 # make sure the bid is not
        //          processed again
        bid.amount = 0;
        //          # some ask remains so go to the next bid
        //          continue
        continue;
    }
}
}
}
return sales;
}

```

OrderBook.h:

```

#pragma once
#include "CSVReader.h"
#include "Candlestick.h"
#include "OrderBookEntry.h"
#include "Volume.h"
#include <string>
#include <vector>

class OrderBook {

```

```

private:
    std::vector<OrderBookEntry> orders;

public:
    /** construct, reading a csv data file */
    OrderBook(std::string filename);
    /** return vector of all know products in the dataset*/
    std::vector<std::string> getKnownProducts();
    /** return vector of Orders according to the sent filters*/
    std::vector<OrderBookEntry> getOrders(OrderBookType type, std::string product,
                                           std::string timestamp);

    /** returns the earliest time in the orderbook*/
    std::string getEarliestTime();
    /** returns the next time after the
     * sent time in the orderbook
     * If there is no next timestamp, wraps around to the start
     */
    std::string getNextTime(std::string timestamp);

    /** returns the last time before the
     * sent time in the orderbook
     * If there is no last timestamp, return empty string
     */
    std::string getLastTime(std::string timestamp);

    void insertOrder(OrderBookEntry &order);

    std::vector<OrderBookEntry> matchAsksToBids(std::string product,
                                                std::string timestamp);

    static long double getHighPrice(std::vector<OrderBookEntry> &orders);
    static long double getLowPrice(std::vector<OrderBookEntry> &orders);
    static long double
    getOpeningAndClosingPrice(std::vector<OrderBookEntry> &orders);
    std::vector<Candlestick> computeCandlesticks(std::string product,
                                                  std::string type,
                                                  std::string currentTime);

    std::vector<std::string> getIndividualProducts();
    std::vector<std::string> getSecondaryProducts();
    std::vector<Volume> computeVolumes(std::string currentTime);
    double averagePrice(std::string product, std::string timestamp);
};

OrderBookEntry.cpp:

```

```

#include "OrderBookEntry.h"

OrderBookEntry::OrderBookEntry(double _price, double _amount,
                                std::string _timestamp, std::string _product,
                                OrderBookType _orderType, std::string _username)
    : price(_price), amount(_amount), timestamp(_timestamp), product(_product),
      orderType(_orderType), username(_username) {}

OrderBookType OrderBookEntry::stringToOrderBookType(std::string s) {
    if (s == "ask") {
        return OrderBookType::ask;
    }
    if (s == "bid") {
        return OrderBookType::bid;
    }
    return OrderBookType::unknown;
}

OrderBookEntry.h:

#pragma once

#include <string>

enum class OrderBookType { bid, ask, unknown, asksale, bidsale };

class OrderBookEntry {
public:
    OrderBookEntry(double _price, double _amount, std::string _timestamp,
                   std::string _product, OrderBookType _orderType,
                   std::string username = "dataset");

    static OrderBookType stringToOrderBookType(std::string s);

    static bool compareByTimestamp(OrderBookEntry &e1, OrderBookEntry &e2) {
        return e1.timestamp < e2.timestamp;
    }
    static bool compareByPriceAsc(OrderBookEntry &e1, OrderBookEntry &e2) {
        return e1.price < e2.price;
    }
    static bool compareByPriceDesc(OrderBookEntry &e1, OrderBookEntry &e2) {
        return e1.price > e2.price;
    }

    double price;
    double amount;
    std::string timestamp;

```

```

        std::string product;
        OrderBookType orderType;
        std::string username;
    };

```

Volume.cpp:

```

// ADDITION #22 - VOLUME CLASS IMPLEMENTATION FILE
#include "Volume.h"

// Constructor
Volume::Volume(std::string product, long double volume)
    : product(product), volume(volume) {}

// Getters
std::string Volume::getProduct() { return product; }
long double Volume::getVolume() { return volume; }

// Setters
void Volume::setProduct(std::string product) { this->product = product; }
void Volume::setVolume(long double volume) { this->volume = volume; }

// Static methods
/**@param volumes The vector of volumes
 * @return The total volume of all the volumes in the vector
 */
long double Volume::getTotalVolume(std::vector<Volume> &volumes) {
    long double totalVolume = 0;
    for (Volume volume : volumes) {
        totalVolume += volume.getVolume();
    }
    return totalVolume;
}
// END OF STUDENT CODE

```

Volume.h:

```

// ADDITION #21 - VOLUME CLASS HEADER FILE
// This is the header file for the Volume class, which is used to store a single
// volume of a product. It also included a static method to get the total volume
// of a vector of Volume objects, aside from the usual getters and setters and
// constructor.
#pragma once

#include <string>
#include <vector>

class Volume {

```

```

private:
    std::string product;
    long double volume;

public:
    // constructors
    Volume(std::string product, long double volume);

    // getters
    std::string getProduct();
    long double getVolume();

    // setters
    void setProduct(std::string product);
    void setVolume(long double volume);

    // static methods
    static long double getTotalVolume(std::vector<Volume> &volumes);
};
// END OF STUDENT CODE

VolumeGraph.cpp:

// ADDITION #24 - VOLUMEGRAPH CLASS IMPLEMENTATION FILE
// --- SATISFYING TASK 3 REQUIREMENTS ---
#include "VolumeGraph.h"
#include "Helpers.h"
#include <cmath>

// constructor
VolumeGraph::VolumeGraph(std::vector<Volume> entries, int width, int height,
                          int xEntryLength, int xEntryCount, int yEntryLength,
                          int yEntryCount)
    : Canvas(width, height), entries(entries), X_ENTRY_LENGTH(xEntryLength),
      X_ENTRY_COUNT(entries.size()), Y_ENTRY_LENGTH(yEntryLength),
      Y_ENTRY_COUNT(yEntryCount), PADDING_LEFT(width / 10),
      PADDING_BOTTOM(height / 10), maxPointY{PADDING_LEFT, 0},
      minPointY(PADDING_LEFT, 0),
      X_INTERVAL((width - (width / 10)) / (xEntryCount + 1)),
      Y_INTERVAL(height / yEntryCount) {}

// getters
std::vector<Volume> VolumeGraph::getEntries() const { return entries; }
int VolumeGraph::getXEntryLength() const { return X_ENTRY_LENGTH; }
int VolumeGraph::getYEntryLength() const { return Y_ENTRY_LENGTH; }
int VolumeGraph::getXEntryCount() const { return X_ENTRY_COUNT; }
int VolumeGraph::getYEntryCount() const { return Y_ENTRY_COUNT; }

```

```

int VolumeGraph::getXInterval() const { return X_INTERVAL; }
int VolumeGraph::getYInterval() const { return Y_INTERVAL; }
int VolumeGraph::getPaddingLeft() const { return PADDING_LEFT; }
int VolumeGraph::getPaddingBottom() const { return PADDING_BOTTOM; }

// setters
/** Functions to set canvas pixels to be a raw Cartesian system consists of x
 * and y axes*/
void VolumeGraph::setXAxis() {
    setHorizontalLine(0, WIDTH, HEIGHT - PADDING_BOTTOM, "-");
}

void VolumeGraph::setYAxis() { setVerticalLine(0, HEIGHT, PADDING_LEFT, "|"); }

/** Function to set Y axis, including Y entries, markers, and also a bit of code
 * to set the correct minPoint on the Y axis (aka the lowest marker on the Y
 * axis)*/
void VolumeGraph::completeYAxis() {
    for (int y = 0; y < HEIGHT - PADDING_BOTTOM; y += Y_INTERVAL) {
        long double totalVolume = Volume::getTotalVolume(entries);
        int index = y / Y_INTERVAL;
        setPixel(PADDING_LEFT, y, ""); // draw Y interval

        // draw Y entries
        std::string text =
            shortenNumber(totalVolume - (totalVolume / Y_ENTRY_COUNT) * index);
        // prepend to the text with spaces if the text is shorter than the length
        // of the Y entries
        if (text.length() < Y_ENTRY_LENGTH) {
            text = std::string(Y_ENTRY_LENGTH - text.length(), ' ') + text;
        }
        for (int x = 0; x < Y_ENTRY_LENGTH; x++) {
            setPixel(x, y, typewriter(x, text)); // write Y entries
        }

        // update min point on Y axis
        if (y == (Y_ENTRY_COUNT - 1) * Y_INTERVAL) {
            minPointY.setY(y);
        }
    }
}

/** Function to set X axis, including X entries, markers*/
void VolumeGraph::completeXAxis() {
    for (int x = PADDING_LEFT + X_INTERVAL; x < WIDTH; x += X_INTERVAL) {

```

```

// get index
int index = (x - (PADDING_LEFT + X_INTERVAL)) / X_INTERVAL;

if (index < entries.size()) {
    setPixel(x, HEIGHT - PADDING_BOTTOM, " "); // draw X interval
    xAxisPoints.push_back(Coord(x, HEIGHT - PADDING_BOTTOM));

    std::string text = entries[index].getProduct();

    for (int j = 0; j < text.size(); j++) {
        setPixel(x + j - 1, xAxisPoints[index].getY() + 1,
            typewriter(j, text)); // draw X entries
    }
}
}

/** Function to set the volume graph, including the percentage of each volume
 * entry*/
void VolumeGraph::setVolumeGraph() {
    // get total volume of all entries
    long double totalVolume = Volume::getTotalVolume(entries);

    for (int i = 0; i < xAxisPoints.size(); i++) {
        int xAnchor = xAxisPoints[i].getX();
        // some convenience variables to make the code more readable
        int yFoot = HEIGHT - PADDING_BOTTOM - 1;
        std::vector<long double> drawVector = processMappingUpper(
            mapping(entries[i].getVolume(), 0, totalVolume, 0,
                yFoot)); // get y location of each entry by using the helper
            // mapping function
        int yLoc = drawVector[0];
        int yRemain = drawVector[1];
        int yHead = HEIGHT - PADDING_BOTTOM - yLoc - 1;
        std::string sym = processRemainder(yRemain, "upper");
        std::string fullSym = "\033[33;41m" + sym.append("\033[0m");
        // get percentage of each entry, creating the entry text
        long double percentage = entries[i].getVolume() / totalVolume * 100;
        std::string text = std::to_string(percentage) + "%";
        // draw the columns, width 5
        setColumn(yFoot, maxPointY.getY(), xAnchor, colorize(" ", "red"),
            5); // background column in red
        // We have 4 cases here:
        // 1. yLoc > 0 and no remainder
        if (yLoc > 0 && yRemain == 0)
            // draw the column with the full symbol

```

```

        setColumn(yFoot, yHead, xAnchor, colorize(" ", "yellow"), 5);
// 2. yLoc > 0 and there is remainder
else if (yLoc > 0 && yRemain > 0) {
    // draw the column with the full symbol
    setColumn(yFoot, yHead, xAnchor, colorize(" ", "yellow"), 5);
    // then draw the remainder on top
    setColumn(yHead - 1, yHead - 1, xAnchor, fullSym, 5);
}
// 3/ yLoc == 0 and there is remainder
else if (yLoc == 0 && yRemain > 0) {
    // no full column, just draw the remainder
    setColumn(yFoot, yHead, xAnchor, fullSym, 5);
}
// draw the entry text (names of the product)
for (int j = 0; j < text.size(); j++) {
    setPixel(xAnchor + j + 3, yHead, typewriter(j, text));
}
// draw the volume text
std::string volumeText = shortenNumber(entries[i].getVolume());
for (int j = 0; j < volumeText.size(); j++) {
    setPixel(xAnchor + j - (volumeText.size() + 2), yHead,
            typewriter(j, volumeText));
}
}
}

/** Function to bundle every neccessary actions to prepare a CandlestickGraph,
 * from nothing to being ready to be drawn to the canvas, as I don't want to
 * call these 5 methods everytime, cluttering up the code in main*/
void VolumeGraph::prep() {
    setXAxis();
    setYAxis();
    completeYAxis();
    completeXAxis();
    setVolumeGraph();
}
// END OF STUDENT CODE

VolumeGraph.h:

// ADDITION #23 - VOLUMEGRAPH CLASS HEADER FILE
#pragma once

#include "Canvas.h"
#include "Coord.h"
#include "Helpers.h"
#include "Volume.h"

```

```

// This class is another child of the Canvas class, which shares all the public
// getters and setters and print methods. It can also access protected members
// such as width or height of the canvas. As a sibling of the CandlestickGraph
// class, it has many similarities with the elder sibling, yet is much more
// concise and considerably simpler.
class VolumeGraph : public Canvas {
private:
    std::vector<Volume> entries;

    const int X_ENTRY_LENGTH;
    const int Y_ENTRY_LENGTH;
    const int X_ENTRY_COUNT;
    const int Y_ENTRY_COUNT;
    const int X_INTERVAL;
    const int Y_INTERVAL;
    const int PADDING_LEFT;
    const int PADDING_BOTTOM;

    std::vector<Coord> xAxisPoints;

    Coord maxPointY;
    Coord minPointY;

public:
    // constructor - with all defaults defined
    VolumeGraph(std::vector<Volume> entries, int width = 132, int height = 33,
                int xEntryLength = 10, int xEntryCount = 3, int yEntryLength = 12,
                int yEntryCount = 10);

    // getters
    std::vector<Volume> getEntries() const;
    int getXEntryLength() const;
    int getYEntryLength() const;
    int getXEntryCount() const;
    int getYEntryCount() const;
    int getXInterval() const;
    int getYInterval() const;
    int getPaddingLeft() const;
    int getPaddingBottom() const;

    // setters
    void setXAxis();
    void setYAxis();
    void completeYAxis();
    void completeXAxis();

```

```

        void setVolumeGraph();

        // other methods
        void prep();
    };
    // END OF STUDENT CODE

Wallet.cpp:

#include "Wallet.h"
#include <iostream>
#include "CSVReader.h"

Wallet::Wallet()
{

}

void Wallet::insertCurrency(std::string type, double amount)
{
    double balance;
    if (amount < 0)
    {
        throw std::exception{};
    }
    if (currencies.count(type) == 0) // not there yet
    {
        balance = 0;
    }
    else { // is there
        balance = currencies[type];
    }
    balance += amount;
    currencies[type] = balance;
}

bool Wallet::removeCurrency(std::string type, double amount)
{
    if (amount < 0)
    {
        return false;
    }
    if (currencies.count(type) == 0) // not there yet
    {
        //std::cout << "No currency for " << type << std::endl;
        return false;
    }
}

```

```

    }
    else { // is there - do we have enough
        if (containsCurrency(type, amount)) // we have enough
        {
            //std::cout << "Removing " << type << ": " << amount << std::endl;
            currencies[type] -= amount;
            return true;
        }
        else // they have it but not enough.
            return false;
    }
}

bool Wallet::containsCurrency(std::string type, double amount)
{
    if (currencies.count(type) == 0) // not there yet
        return false;
    else
        return currencies[type] >= amount;
}

std::string Wallet::toString()
{
    std::string s;
    for (std::pair<std::string, double> pair : currencies)
    {
        std::string currency = pair.first;
        double amount = pair.second;
        s += currency + " : " + std::to_string(amount) + "\n";
    }
    return s;
}

bool Wallet::canFulfillOrder(OrderBookEntry order)
{
    std::vector<std::string> currs = CSVReader::tokenise(order.product, '/');
    // ask
    if (order.orderType == OrderBookType::ask)
    {
        double amount = order.amount;
        std::string currency = currs[0];
        std::cout << "Wallet::canFulfillOrder " << currency << " : " << amount << std::endl;

        return containsCurrency(currency, amount);
    }
}

```

```

        // bid
        if (order.orderType == OrderBookType::bid)
        {
            double amount = order.amount * order.price;
            std::string currency = currs[1];
            std::cout << "Wallet::canFulfillOrder " << currency << " : " << amount << std::endl;
            return containsCurrency(currency, amount);
        }

        return false;
    }

void Wallet::processSale(OrderBookEntry& sale)
{
    std::vector<std::string> currs = CSVReader::tokenise(sale.product, '/');
    // ask
    if (sale.orderType == OrderBookType::asksale)
    {
        double outgoingAmount = sale.amount;
        std::string outgoingCurrency = currs[0];
        double incomingAmount = sale.amount * sale.price;
        std::string incomingCurrency = currs[1];

        currencies[incomingCurrency] += incomingAmount;
        currencies[outgoingCurrency] -= outgoingAmount;
    }
    // bid
    if (sale.orderType == OrderBookType::bidsale)
    {
        double incomingAmount = sale.amount;
        std::string incomingCurrency = currs[0];
        double outgoingAmount = sale.amount * sale.price;
        std::string outgoingCurrency = currs[1];

        currencies[incomingCurrency] += incomingAmount;
        currencies[outgoingCurrency] -= outgoingAmount;
    }
}

std::ostream& operator<<(std::ostream& os, Wallet& wallet)
{
    os << wallet.toString();
    return os;
}

```

Wallet.h:

```
#pragma once

#include "OrderBookEntry.h"
#include <iostream>
#include <map>
#include <string>

class Wallet {
public:
    Wallet();
    /** insert currency to the wallet */
    void insertCurrency(std::string type, double amount);
    /** remove currency from the wallet */
    bool removeCurrency(std::string type, double amount);

    /** check if the wallet contains this much currency or more */
    bool containsCurrency(std::string type, double amount);
    /** checks if the wallet can cope with this ask or bid.*/
    bool canFulfillOrder(OrderBookEntry order);
    /** update the contents of the wallet
     * assumes the order was made by the owner of the wallet
     */
    void processSale(OrderBookEntry &sale);

    /** generate a string representation of the wallet */
    std::string toString();
    friend std::ostream &operator<<(std::ostream &os, Wallet &wallet);

private:
    std::map<std::string, double> currencies;
};
```